

CLARO ARGENTINA · ESTACIÓN INCLUSIVA

# Diagnóstico Técnico y Plan de Mejoras

Análisis integral de producto, UX/UI y arquitectura — con plan de implementación priorizado por impacto.

Documento interno · Confidencial

PROYECTO

Claro LSA — App de Atención Inclusiva

FECHA

25 de agosto de 2026

ESTADO DEL PROYECTO

Beta funcional

TOTAL DE ITEMS ANALIZADOS

12 correcciones + hallazgos propios

1

Resumen del Proyecto

**Claro Estación Inclusiva** es una PWA (Progressive Web App) diseñada para facilitar la comunicación entre personas sordas y agentes de atención al cliente en oficinas presenciales de Claro Argentina. Opera bajo un **sistema de turnos (pass-the-phone)**: cliente sordo y agente se pasan el dispositivo alternadamente para comunicarse usando señas, texto grande o audio sintetizado.

STACK TECNOLÓGICO

React 18TypeScript 5Vite 5Tailwind CSS 3PWAPWA

INTEGRACIONES EXTERNAS

Dillo LSA (iframe)Dillo Avatar (iframe)Speech Synthesis APIdillo-interpreter plugin

PANTALLAS ACTUALES

3 pantallas + 5 overlays

Bienvenida → Volumen → Servicios

MODOS DE COMUNICACIÓN

3 modos

Mis manos · Responder con Dillo · Responder con texto

Flujo actual de la aplicación

1. Pantalla de Bienvenida (logo + botón "Comenzar") → 2. Recordatorio de Volumen → 3. Pantalla Principal: grilla de 9 Toques Rápidos + 3 botones de modo de conversación.

**Flujo Toque Rápido:** cliente selecciona servicio → (opc.) selecciona subopción → (opc.) ingresa teléfono/monto → mensaje en pantalla grande + TTS → botón "Listo".

**Flujo Seña/Dillo/Texto:** cliente presiona modo → se abre IframeModal con tabs para cambiar entre modos.

Evaluación general del estado actual

4

Críticos

7

Alto impacto

4

Bajo / Limitado

## 2

## Análisis de las 12 Correcciones — Prioridad Crítica

### #2 · Respuestas ida y vuelta en una misma pantalla

CRÍTICO

**Problema:** Actualmente, cuando el cliente hace un Toque Rápido, la app muestra el mensaje en QuickTouchScreen con un botón "Listo" que cierra la pantalla y vuelve al inicio. El agente no tiene forma de responder desde esa misma vista — tiene que devolver el teléfono, que el cliente vuelva a seleccionar algo, etc. El flujo de conversación es puramente unidireccional.

`QuickTouchScreen.tsx:171-179` — El botón "Listo" solo llama a `onClose()`, que en `ServiceSelectionScreen` setea `activeQuickTouch = null`. No hay forma de responder.

#### ✓ Solución propuesta

Transformar QuickTouchScreen en una pantalla de conversación de dos turnos. Después de que el cliente muestra el mensaje, aparece un botón "Responder" (en lugar de "Listo") que despliega opciones de respuesta del agente (texto, Dillo, seña) en la misma pantalla, sin cerrar el modal. El historial del turno queda visible mientras el agente responde.

#### ⚖ Trade-off

Requiere refactorizar QuickTouchScreen para manejar dos fases: "vista del cliente" y "respuesta del agente". Aumenta la complejidad del componente (~+100 líneas) pero elimina la mayor fricción del flujo actual.

### #3 · Reemplazar el botón "Listo" por "Responder"

CRÍTICO

**Problema:** El botón "Listo" ( `QuickTouchScreen.tsx:171` ) comunica cierre, no continuidad. En un sistema de turnos, el agente necesita poder *tomar el turno* y responder, no cerrar la interacción.

El mismo componente se reutiliza en `TextResponseMode` con `closeLabel="Escribir otro mensaje"`, lo que demuestra que el sistema ya soporta labels custom — falta extenderlo para el caso de respuesta.

#### ✓ Solución propuesta

Reemplazar el botón primario por "Responder al asesor" que abre un mini-panel de respuesta incrustado (texto libre / abrir Dillo / abrir seña), sin cerrar la vista del mensaje. El botón "Listo" pasa a ser secundario y solo aparece una vez que terminó el intercambio completo.

## #12 · Sistema de turnos entre agente y persona sorda

CRÍTICO

**Problema:** No existe ningún indicador visual de a quién le toca interactuar. En un entorno de atención presencial con ruido y presión, la ambigüedad genera confusión: ¿quién tiene el teléfono? ¿el agente ya terminó? ¿el cliente ya leyó? El flujo implícito no es suficiente.

### ✓ Solución propuesta

Agregar un banner de turno en la parte superior de la pantalla, con dos estados visuales claros:

■ **"Tu turno — hacé tu consulta"**

(fondo rojo, para el cliente) y

■ **"Turno del asesor — esperá la respuesta"**

(fondo azul, para cuando tiene el teléfono el agente). El turno cambia al presionar "Responder" o "Listo".

### ⚖ Trade-off

Necesita coordinación con #2 y #3 para que los turnos cambien de forma coherente. Si se implementa solo, sin #2 y #3, el indicador de turno no tiene acción asociada y resulta confuso.

## #11 · Reactivar el scroll en smartphones

CRÍTICO

**Problema:** `QuickTouchScreen.tsx:19-32` aplica `body.style.overflow = 'hidden'` al montar el componente. Esto bloquea el scroll de TODA la página mientras QuickTouchScreen está activo. En teléfonos con pantalla pequeña (<640px), el contenido queda cortado y no es accesible.

Adicionalmente, el contenedor interno tiene `overflow-y-auto` pero la altura puede no calcularse correctamente en todos los dispositivos cuando el cuerpo está bloqueado.

### ✓ Solución propuesta

Eliminar el bloqueo de `body.overflow` en QuickTouchScreen. La pantalla fullscreen ( `fixed inset-0` ) ya aísla el scroll nativo. Agregar `overflow-y-auto` y `-webkit-overflow-scrolling: touch` en el contenedor interno. El overscroll-behavior en el root puede mantenerse para prevenir pull-to-refresh.

### 3 Correcciones — Alto Impacto

#### #4 · Botón de respuesta en Toques Rápidos

**ALTO IMPACTO**

**Problema:** La grilla de 9 Toques Rápidos ( `ServiceSelectionScreen.tsx:382-403` ) no tiene una vía directa para que el agente responda sin que el cliente tenga que seleccionar algo primero. El agente recibe el mensaje pero no puede contestar desde esa sección.

##### ✓ Solución propuesta

Agregar una barra de acciones debajo de la grilla de Toques Rápidos con un botón "El asesor responde →" que abre el `IframeModal` en modo texto directamente. Esta barra solo es visible cuando corresponde al turno del agente (se coordina con #12).

##### ⚠ Trade-off

Si no se implementa el sistema de turnos (#12), este botón puede confundir: el cliente podría presionarlo por error. Implementar con #12 para que sea contextual.

#### #5 · Botón "Responder al asesor" en pantalla de detección de seña

**ALTO IMPACTO**

**Problema:** En `IframeModal` ( `IframeModal.tsx:100-112` ), cuando el cliente hace la seña y la pasa al agente, el agente solo ve el iframe de Dillo LSA mostrando la traducción. No tiene un botón claro para "tomar el turno" y responder desde esa misma pantalla.

El agente actualmente tendría que cerrar el modal (botón "Inicio") y volver a la pantalla principal para responder, lo cual pierde el contexto de la conversación.

##### ✓ Solución propuesta

Agregar un botón flotante o de overlay

##### "Responder al asesor"

que aparece dentro del `IframeModal` modo "hands", visible para el agente. Al presionarlo, el modal cambia al tab de "Responder con texto" o "Responder con Dillo" sin cerrar la sesión. Mantiene el hilo de conversación dentro del mismo modal.

## #1 · Video de bienvenida + indicaciones de uso

ALTO IMPACTO

FEATURE NUEVA

**Problema:** La pantalla de bienvenida actual ( `WelcomeScreen.tsx` ) solo tiene un logo y un botón "Comenzar". Para usuarios nuevos (tanto personas sordas como agentes que nunca usaron la app), no hay instrucción visual de cómo funciona el sistema de turnos.

La pantalla de VolumeReminderScreen da una instrucción puntual (volumen), pero no el flujo completo de uso.

### ✓ Solución propuesta — 3 opciones rankeadas

#### Opción A (Recomendada):

Agregar un video corto (30-45 seg) que se reproduce automáticamente al entrar a WelcomeScreen, con subtítulos quemados. El video se aloja externamente (YouTube unlisted o CDN propio). Botón "Ver video" opcional y botón "Saltar" para usuarios que ya conocen el sistema.

#### Opción B:

Tutorial tipo carrusel (3-4 slides) con imágenes e instrucciones de texto. Más liviano, sin dependencia externa.

#### Opción C:

Mantener HelpModal actual como tutorial de primer uso, mostrándolo automáticamente la primera vez (localStorage flag). Mínimo cambio.

### ⚠ Consideración

El video requiere producción externa (grabación, edición, hosting). Si no hay presupuesto para video inmediato, Opción C es deployable hoy. Opción A tiene el mayor impacto de adopción pero requiere 1-2 semanas de producción.

## #10 · Historial de conversación por sesión

ALTO IMPACTO

FEATURE NUEVA

**Problema:** No hay registro de lo que se comunicó durante una sesión. Si el agente necesita revisar qué le dijo el cliente anteriormente (por ejemplo, al hacer una gestión que requiere confirmar datos), no puede. Los Toques Rápidos y mensajes de texto se pierden al cerrar las pantallas.

### ✓ Solución propuesta

Agregar un store de mensajes en App.tsx (useState con array). Cada mensaje capturado (Toque Rápido, texto libre) se agrega al array con timestamp y rol (cliente/agente). Un botón en el header de ServiceSelectionScreen abre un drawer lateral con el historial de la sesión. Al cerrar la app o presionar "Nueva atención", el historial se limpia.

### ⚠ Limitación técnica

Los mensajes de los iframes (señas traducidas por Dillo LSA, respuestas del Avatar) *no son capturables* sin una API de Dillo que exponga el texto traducido vía postMessage. El historial solo puede capturar lo que pasa en la app nativa (toques rápidos, texto escrito). Mencionarlo en el alcance.

## 4

## Correcciones — Medio Impacto y Técnicamente Limitadas

## #6 · Letras más oscuras en la traducción

MEDIO

**Contexto doble:** Este ítem puede referirse a (a) el texto nativo en QuickTouchScreen, o (b) el texto dentro del iframe de Dillo LSA.

**(a) Texto nativo:** El título en QuickTouchScreen usa `text- [#DA291C]` (rojo Claro) sobre `bg-gray-50`. Ratio de contraste ~3.7:1 — pasa para texto grande pero está en el límite de WCAG AA.

**(b) Texto del iframe:** El texto de traducción que genera Dillo LSA está dentro de un iframe de `entrenar.dillo.ar` — no podemos modificarlo directamente desde esta app. Solo sería posible si Dillo expone una API de tematización.

## ✓ Solución para (a)

Cambiar el color del título en QuickTouchScreen de `#DA291C` a `#111827` (casi negro). El borde rojo del card mantiene la identidad visual. Ratio de contraste pasa a 16:1. Cambio de 2 líneas.

## ⚠ Para (b)

No controlable sin cambios del lado de Dillo. Requiere solicitud formal al equipo de Dillo para exponer tematización del iframe.

## #9 · Eliminar el widget del avatar

BAJO ESFUERZO

**Situación actual:** El widget `<dillo-avatar-widget lang="lsa">` se monta en `App.tsx:138` cuando `avatarOn === true`. El switch de control está en el header de `ServiceSelectionScreen`. El estado se elevó a `App.tsx` en el último commit.

## ✓ Solución — cambios específicos

1. Eliminar la línea `App.tsx:138` que monta el widget.
  2. Eliminar el estado `avatarOn` y `onToggleAvatar` de `App.tsx`.
  3. Eliminar las props `avatarOn` y `onToggleAvatar` de `ServiceSelectionScreenProps`.
  4. Eliminar el switch button del header de `ServiceSelectionScreen`.
- Tiempo estimado: 15 minutos.

## #7 · Micrófono activado por default

**TÉCNICAMENTE LIMITADO**

**Situación:** El modo "Mis manos" carga un iframe de `entrenar.dillo.ar/#/traductor`. El micrófono lo activa el propio iframe internamente. Desde esta app solo podemos pasar parámetros en la URL o enviar mensajes via `postMessage`.

### ⚠ Limitación

Sin documentación pública de la API del iframe de `entrenar.dillo.ar`, no podemos activar el micrófono de forma programática. Si se agrega el param `?autostart=1` a la URL no hay garantía de que funcione.

### Requiere confirmación del equipo de Dillo

sobre si el iframe expone esta capacidad.

### ✓ Lo que sí podemos hacer

Intentar `getHandsUrl()` con param `&autoMic=true` o similar. Si Dillo lo soporta, el cambio es trivial (1 línea). Paralelamente, solicitar la documentación de la API del iframe a Dillo.

## #8 · Audio del intérprete activado por default

**TÉCNICAMENTE LIMITADO**

**Situación:** El avatar de Dillo carga desde `https://avatar.dillo.ai/?embed=1`. El parámetro `?embed=1` ya configura el iframe para uso embebido. El audio del intérprete lo controla el propio iframe.

### ⚠ Limitación

Misma limitación que #7: el iframe es de dominio externo. Los navegadores modernos bloquean el autoplay de audio sin interacción del usuario (política de Autoplay). Aunque Dillo lo soporte vía `postMessage`, el primer audio probablemente requiera un tap del usuario de todas formas (restricción de Safari/Chrome móvil).

### ✓ Lo que sí podemos hacer

Intentar `DILLO_URL = 'https://avatar.dillo.ai/?embed=1&autoplay=1'`. Probar en dispositivos reales. Para el autoplay de audio: agregar un botón "Activar audio" visible cuando el avatar carga, que el usuario presiona una vez para desbloquear el contexto de audio del iframe.

## 5 Hallazgos Propios del Análisis (no mencionados en las correcciones)

### [P1] Indicadores "Cámara lista" y "Aplicación lista" son siempre falsos

CRÍTICO · UX

`WelcomeScreen.tsx:43-52` — Dos puntos verdes hardcodeados con `animate-pulse`. La cámara nunca se verifica. Si el dispositivo no tiene cámara habilitada o el permiso fue denegado, el indicador sigue verde. Genera desconfianza y puede derivar en problemas de uso del modo "Mis manos".

#### ✓ Solución

Reemplazar con un check real: `navigator.mediaDevices.getUserMedia({video: true})`. Si el permiso es concedido → verde. Si es denegado → rojo con mensaje "Activá la cámara en el navegador". Opcionalmente, mostrar "Verificando..." mientras espera.

### [P2] Cambiar de tab en IframeModal recarga el iframe completo

CRÍTICO · PERFORMANCE

`IframeModal.tsx:105-106` — `key={iframeSrc}` hace que React destruya y recree el iframe cada vez que el usuario cambia entre "Mis manos" y "Responder con Dillo". La cámara se corta, el estado de Dillo se pierde y la carga puede tomar 3-8 segundos.

#### ✓ Solución

Montar *ambos* iframes al abrir el modal y alternar visibilidad con CSS `display: none / block`. El iframe de señas sigue cargado cuando el usuario cambia a Dillo, y viceversa. El timestamp de la URL de LSA se sigue generando una sola vez al montar.

### [P3] No hay estado de carga (loading) en los iframes

ALTO · UX

Al abrir IframeModal, los iframes pueden tardar 3-8 segundos en cargar. El usuario ve pantalla blanca sin ningún indicador. En un entorno de atención presencial con el agente esperando, esto genera tensión y confusión.

#### ✓ Solución

Agregar un spinner o skeleton sobre el iframe, ocultarlo en el evento `onLoad` del iframe. 10 líneas de código, alto impacto percibido.

### [P4] Texto de 11px en la grilla de Toques Rápidos — viola WCAG AA

ALTO · ACCESIBILIDAD

`ServiceSelectionScreen.tsx:395` — `text-[11px]` en mobile para los labels de los botones. WCAG AA requiere mínimo 14px para texto de interfaz interactivo. Una app de inclusión para personas con discapacidad no puede fallar en accesibilidad básica.

#### ✓ Solución

Cambiar `text-[11px]` a `text-xs` (12px mínimo) o `text-[13px]`. Si el texto no entra, reducir el padding o usar `text-ellipsis overflow-hidden`. Cambio de 1 línea.

**[P5] Carpetas "backup/" y "build dev/" commiteadas al repositorio****ALTO · TÉCNICO**

El último commit ( `b4ca5b9` ) agrega 45 archivos que incluyen dos builds completos de la app ( `backup/` y `build dev/` ) y un PDF binario. Los builds compilados en git: (1) no tienen control de versiones útil, (2) engordan el repo ~8MB, (3) pueden incluir código desactualizado que confunde a otros devs.

**✓ Solución**

Agregar `backup/` , `build dev/` y `dist-dev/` al `.gitignore` . Los builds de producción ( `dist/` ) tampoco deberían estar en el repo si hay CI/CD. Limpiar historial con `git rm --cached` .

**[P6] Supabase instalado pero sin ningún uso****MEDIO · TÉCNICO**

`package.json:dependencies` — `@supabase/supabase-js ^2.57.4` está declarado pero no hay ningún import en el codebase. Añade ~150KB al bundle de producción sin valor. Si se va a usar para analytics o historial (#10), documentarlo; si no, removerlo.

**✓ Solución**

`npm uninstall @supabase/supabase-js` . Si el plan es usarlo para el historial (#10), mantenerlo y documentar la intención en el README.

**[P7] Logos de Claro cargados desde CDN externo — punto de fallo****MEDIO · CONFIABILIDAD**

`WelcomeScreen.tsx:16` , `HelpModal.tsx:13` — Logos cargados desde `https://www.claro.com.ar/static/` . Si el dominio tiene un problema puntual (mantenimiento, CDN caído), la app aparece sin logo. En un entorno de atención al cliente esto es inaceptable.

**✓ Solución**

Descargar los SVGs de Claro y colocarlos en `public/icons/` . Referenciarlos como `src="/claro/icons/claro-logo.svg"` . 5 minutos de trabajo.

**[P8] "Traducir Página web" en el HelpModal no aplica a esta app****BAJO · UX**

`HelpModal.tsx:83-88` — La opción 4 del tutorial explica cómo seleccionar texto de una página web para traducirlo con Dillo. Esta funcionalidad no existe en esta SPA. Confunde al usuario sobre qué puede y no puede hacer la app.

**✓ Solución**

Reemplazar el ítem 4 por una descripción del modo "Responder con texto" (que sí existe). Cambio de texto en `HelpModal.tsx`.

## 6 Roadmap de Implementación

### Sprint 0

#### Deuda técnica y quick wins

🕒 1-2 días

- 1 [P5] Limpiar backup/ y build dev/ del repo + .gitignore  
Previene confusión y engordar del repo. Riesgo: ninguno.
- 2 #9 Eliminar widget del avatar (switch + estado + componente)  
3 archivos, ~15 líneas a eliminar. Sin impacto en flujo principal.
- 3 [P7] Bajar logos de Claro a assets locales  
Elimina dependencia externa crítica. 5 minutos.
- 4 [P6] Desinstalar Supabase (si no se va a usar ahora)  
Reduce bundle ~150KB. 1 comando.
- 5 [P4] Corregir text-[11px] → mínimo text-xs en Toques Rápidos  
WCAG AA. 1 línea.
- 6 #6 Cambiar color de título en QuickTouchScreen a negro  
Mejora contraste WCAG. 1 línea.
- 7 [P8] Corregir ítem 4 del HelpModal  
Reemplazar "Traducir Página web" por descripción de "Responder con texto".

### Sprint 1

#### UX crítica del flujo principal

🕒 3-5 días

- 1 #11 Reactivar scroll en smartphones  
Eliminar body.overflow=hidden en QuickTouchScreen. Corregir contenedor interno.
- 2 [P2] Doble iframe en IframeModal con CSS visibility  
Eliminar key={iframeSrc}. Montar ambos iframes al abrir. Elimina la recarga al cambiar tab.
- 3 [P3] Agregar loading state en iframes  
Spinner sobre iframe, se oculta en onLoad. 10 líneas, alto impacto percibido.
- 4 [P1] Verificación real del estado de la cámara en WelcomeScreen  
Usar getUserMedia para check real. Indicador dinámico verde/rojo con mensaje.

## Sprint 2

## Sistema de conversación bidireccional (el cambio más importante)

🕒 1-2 semanas

1

## #2 + #3 Conversación bidireccional en QuickTouchScreen

Transformar la pantalla de mensaje en un módulo de conversación con fases: "vista cliente" → "respuesta asesor". Botón "Responder" en lugar de "Listo".

2

## #12 Sistema de turnos visual

Banner de turno en la parte superior. Se coordina con el flujo bidireccional.

3

## #4 Botón "El asesor responde" en la pantalla de Toques Rápidos

Barra de acciones debajo de la grilla, contextual al turno del asesor.

4

## #5 Botón "Responder al asesor" en pantalla de seña (IframeModal hands)

Botón flotante dentro del modal de señas para que el asesor tome el turno.

## Sprint 3

## Features nuevas de valor alto

🕒 1-2 semanas

1

## #10 Historial de conversación por sesión

Store de mensajes en App.tsx. Panel lateral con timeline. Captura mensajes nativos (no iframes).

2

## #1 Video de bienvenida

Requiere producción externa. Opción C (HelpModal auto en primer uso) deployable mientras tanto.

3

## #7 + #8 Mic y audio por default (pendiente validación con Dillo)

Probar params URL. Solicitar documentación de API a equipo Dillo antes de implementar.

7

Tabla Resumen — Todos los Items

| #                              | Corrección                                             | Prioridad | Esfuerzo           | Sprint   | Archivo/s afectado/s                        |
|--------------------------------|--------------------------------------------------------|-----------|--------------------|----------|---------------------------------------------|
| #2                             | Respuestas ida y vuelta (conversación bidireccional)   | CRÍTICO   | Alto               | Sprint 2 | QuickTouchScreen.tsx, IframeModal.tsx       |
| #3                             | Reemplazar "Listo" por "Responder"                     | CRÍTICO   | Medio              | Sprint 2 | QuickTouchScreen.tsx                        |
| #12                            | Sistema de turnos visual                               | CRÍTICO   | Medio              | Sprint 2 | App.tsx, ServiceSelectionScreen.tsx         |
| #11                            | Reactivar scroll en smartphones                        | CRÍTICO   | Bajo               | Sprint 1 | QuickTouchScreen.tsx                        |
| #4                             | Botón de respuesta en Toques Rápidos                   | ALTO      | Bajo               | Sprint 2 | ServiceSelectionScreen.tsx                  |
| #5                             | Botón "Responder al asesor" en pantalla de señal       | ALTO      | Medio              | Sprint 2 | IframeModal.tsx                             |
| #1                             | Video de bienvenida + indicaciones de uso              | ALTO      | Alto*              | Sprint 3 | WelcomeScreen.tsx (*producción externa)     |
| #10                            | Historial de conversación por sesión                   | ALTO      | Alto               | Sprint 3 | App.tsx + nuevo componente                  |
| #6                             | Letras más oscuras en la traducción                    | MEDIO     | Bajo (a) / N/A (b) | Sprint 0 | QuickTouchScreen.tsx                        |
| #9                             | Eliminar widget del avatar                             | BAJO      | Muy bajo           | Sprint 0 | App.tsx, ServiceSelectionScreen.tsx         |
| #7                             | Micrófono activado por default                         | LIMITADO  | Incierto           | Sprint 3 | IframeModal.tsx (dep. Dillo API)            |
| #8                             | Audio del intérprete por default                       | LIMITADO  | Incierto           | Sprint 3 | IframeModal.tsx (dep. Dillo API + autoplay) |
| Hallazgos propios del análisis |                                                        |           |                    |          |                                             |
| [P1]                           | Indicadores "Cámara lista" falsos → verificación real  | CRÍTICO   | Bajo               | Sprint 1 | WelcomeScreen.tsx                           |
| [P2]                           | Recarga de iframe al cambiar de tab → doble iframe CSS | CRÍTICO   | Medio              | Sprint 1 | IframeModal.tsx                             |
| [P3]                           | Sin loading state en iframes                           | ALTO      | Muy bajo           | Sprint 1 | IframeModal.tsx                             |
| [P4]                           | Texto 11px viola WCAG AA                               | ALTO      | Muy bajo           | Sprint 0 | ServiceSelectionScreen.tsx                  |
| [P5]                           | Carpetas backup/ y build dev/ en el repo               | ALTO      | Muy bajo           | Sprint 0 | .gitignore + git rm --cached                |

| #    | Corrección                                   | Prioridad | Esfuerzo | Sprint   | Archivo/s afectado/s             |
|------|----------------------------------------------|-----------|----------|----------|----------------------------------|
| [P6] | Supabase instalado sin uso                   | MEDIO     | Muy bajo | Sprint 0 | package.json                     |
| [P7] | Logos de Claro desde CDN externo             | MEDIO     | Muy bajo | Sprint 0 | WelcomeScreen.tsx, HelpModal.tsx |
| [P8] | "Traducir Página web" en HelpModal no aplica | BAJO      | Muy bajo | Sprint 0 | HelpModal.tsx                    |

Conclusión y recomendación de próximos pasos

El proyecto tiene una base técnica sólida y un propósito de producto claro. Los problemas más importantes no son de arquitectura sino de **flujo de conversación**: la app actualmente facilita que el cliente *comunique* su necesidad, pero no cierra el loop — el agente no tiene forma de responder desde la misma pantalla. Resolver #2, #3 y #12 transforma la app de una herramienta de comunicación unidireccional a un sistema de conversación real.

**Próximo paso inmediato:** Sprint 0 completo (1-2 días) — limpieza técnica rápida que reduce ruido y deuda antes de entrar al trabajo de UX. Luego Sprint 1 (scroll + iframes), y arrancar el diseño del sistema de conversación bidireccional (#2 + #3 + #12) que es el corazón del Sprint 2.